OXFORDABLE CAREERS · SALESFORCE DEVELOPER INTRODUCTORY PROGRAMME

CareerLaunch

Step-by-Step Build Guide

How to create every part of the Cohort Enrolment & Student Success application: objects and fields, validation rules, flows, Apex, Lightning Web Components, the Enrolment Console page, the Agentforce agent, the permission set, tests and deployment.

GitHub: github.com/dangutdavid/oxfodabledeveloper1_training
Target org: maren.dangut@gmail.com · **Version:** 1.0 · August 2026

> **Already done for you:** everything in this guide has been built, deployed to the org `maren.dangut@gmail.com`, tested (10/10 Apex tests passing) and pushed to the GitHub repository above. Use this guide either to **rebuild each piece by hand** (the best way to learn) or to **understand what was deployed**. Sections 8–9 (page activation and the Agentforce agent) are manual steps you must finish in the org.

Contents

1 · Prerequisites

1. **1.1** A Salesforce Developer Edition org (free, never expires): `developer.salesforce.com/signup`.
2. **1.2** Google Chrome, Visual Studio Code + the **Salesforce Extension Pack**.
3. **1.3** Salesforce CLI — confirm with `sf --version` in a terminal.
4. **1.4** Git installed and a GitHub account.

5. **1.5** In the org: Setup → search **Deliverability** → set Access level to **All Email** (needed for the welcome email in Section 6).

> **Two ways to use this guide:** build every component by hand following Sections 2–11, or reproduce the whole project with your own repo, your own org and an AI assistant using the copy-and-paste runbook in **Appendix B**. The reference source lives at github.com/dangutdavid/oxfodabledeveloper1_training.

2 · The Course object (Lab 1)

2.1 Create the object

1. Log in → gear icon (top right) → **Setup** → **Object Manager** tab → **Create** ▾ → **Custom Object**.
2. Enter the values below, then tick **Launch New Custom Tab Wizard after saving** and click **Save**. In the tab wizard pick any tab style → Next → Next → Save.

Setting	Value
Label / Plural Label	Course / Courses
Object Name	Course
Record Name / Data Type	Course Name / Text
Tick	Allow Reports · Allow Activities · Track Field History

2.2 Add the fields

Object Manager → Course → **Fields & Relationships** → **New**. For each field: choose the type → enter details → leave field-level security defaults (Next) → leave page layout ticked (Save & New).

Field Label	Data Type	Settings	API name created
Course Level	Picklist	Values, one per line: Introductory, Advanced	Course_Level__c
Duration (Weeks)	Number	Length 2, Decimal Places 0	Duration_Weeks__c
Fee	Currency	Length 10, Decimal Places 2	Fee__c
Active	Checkbox	Default: Checked	Active__c
Description	Text Area (Long)	Length 1000	Description__c

3 · The Cohort object (Lab 2)

3.1 Create the object

Setup → Object Manager → Create ▾ → Custom Object:

Setting	Value
Label / Plural	Cohort / Cohorts
Record Name	Cohort Number — Data Type **Auto Number**, Display Format `CO-{0000}`, Starting Number 1
Tick	Allow Reports · Allow Activities · Launch New Custom Tab Wizard

3.2 Add the basic fields (leave the calculated ones for 3.3)

Field	Type	Settings
Course	Lookup Relationship	Related To: **Course**; tick *Required*
Start Date	Date	Tick *Required*
End Date	Date	—
Delivery Mode	Picklist	Online, In-Person, Hybrid
Capacity	Number	Length 3, Decimals 0, Default Value 20, tick *Required*
Status	Picklist	Planned, Open, Full, Running, Completed, Cancelled

> **Do not build Enrolled Count yet** — a roll-up summary is only possible after the master-detail from Enrolment exists (Section 4). Come back to step 3.3 after finishing Section 4.

3.3 Add the three calculated fields (after Section 4)

Field	Type	Definition
Enrolled Count	Roll-Up Summary	Summarised Object: **Enrolments** · Roll-Up Type: **COUNT** · Filter: `Status not equal to Cancelled`
Seats Remaining	Formula → Number (0 dp)	`Capacity__c - Enrolled_Count__c`
Cohort Label	Formula → Text	`Course__r.Name & " - " & TEXT(Start_Date__c)`

3.4 Validation rule

Object Manager → Cohort → **Validation Rules** → New. Name: `End_Date_After_Start_Date`

```
AND(
  NOT(ISBLANK(End_Date__c)),
  End_Date__c < Start_Date__c
)
```

Error message: *End Date cannot be earlier than Start Date.* · Error location: End Date.

4 · The Enrolment object (Lab 2)

> **Spelling matters:** it is **Enrolment** with one “l” (British spelling) everywhere, or the Apex will not compile.

4.1 Create the object

Setting	Value
Label / Plural	Enrolment / Enrolments
Record Name	Enrolment Number — Auto Number, Format `ENR-{00000}`
Tick	Allow Reports · Allow Activities · Launch New Custom Tab Wizard

4.2 Add the fields — in this order (the master-detail must be first)

Field	Type	Settings
Cohort	**Master-Detail** Relationship	Related To: **Cohort**. Accept sharing defaults.
Student	Lookup Relationship	Related To: **Contact**; tick *Required*
Status	Picklist	Applied (set as **default**), Confirmed, Waitlisted, Cancelled, Completed
Enrolment Date	Date	—
Amount Paid	Currency	Length 10, Decimals 2, Default 0
Enrolment Key	Text	Length 64; tick **External ID** and **Unique** (case-insensitive)
Attendance %	Percent	Length 3, Decimals 0
Certificate Issued	Checkbox	Default: Unchecked

4.3 Validation rule

Name: `Payment_Not_Above_Fee` — a cross-object formula spanning two relationships:

```
Amount_Paid__c > Cohort__r.Course__r.Fee__c
```

Error message: *Amount Paid cannot exceed the course fee.*

4.4 Contact field

Object Manager → **Contact** → Fields & Relationships → New → Number, Length 3, Decimals 0, Label **Total Enrolments** (`Total_Enrolments__c`). Maintained automatically by the Queueable job (Section 7).

> **Now go back to step 3.3** and create the roll-up and formulas on Cohort. Then tidy the Cohort page layout: add Enrolled Count, Seats Remaining, Cohort Label, and confirm the Enrolments related list is present.

5 · Sample data

Either create records by hand (Courses tab → New, etc.) or run the seed script from the repo:

```
sf apex run --file scripts/apex/seed.apex --target-org <your-org-alias>
```

Object	Records
Course	Salesforce Developer – Introductory (3 wks, £500) · Salesforce Developer – Advanced (10 wks, £1500)
Cohort	CO-0001: Introductory, starts today+14, **Capacity 3**, Open · CO-0002: Advanced, starts today+30, Capacity 10, Open
Contact	3 students with real-looking email addresses (the agent searches on email)

> **Checkpoint (US-03):** create one Enrolment linking a Contact to CO-0001, then open CO-0001 and confirm **Enrolled Count = 1** and **Seats Remaining = 2** — with no automation at all.

6 · Flows (Lab 3)

6.1 Record-triggered flow — Enrolment_Confirmed_Actions (US-06, US-07)

Setup → search **Flows** → New Flow → **Record-Triggered Flow**:

Setting	Value
Object	Enrolment
Trigger	A record is created or updated
Entry conditions	`Status__c Equals Confirmed` — and "Only when a record is updated to meet the condition requirements"
Optimise for	**Actions and Related Records** (after-save — we touch other records)

Add these elements in order:

1. **Get Records** "Get_Cohort": object Cohort, `Id Equals {!$Record.Cohort__c}`, only first record, fields: Seats_Remaining__c, Status__c, Start_Date__c.
2. **Action** → **Send Email**: Recipient `{!$Record.Student__r.Email}` · Subject `Your place is confirmed – {!$Record.Cohort__r.Cohort_Label__c}` · Body: a text template with the welcome message and start date. **Add a Fault path from this action to the next element** so a failed email never blocks the enrolment save.
3. **Create Records** — a Task: Subject `Send welcome pack to {!$Record.Student__r.Name}`, WhatId `{!$Record.Id}`, OwnerId `{!$Record.Cohort__r.OwnerId}` (the cohort owner — Enrolment is master-detail so it has no owner of its own), ActivityDate `{!$Flow.CurrentDate}`, Priority High.
4. **Decision** "Is Cohort Now Full?": Yes when `{!Get_Cohort.Seats_Remaining__c} <= 0`.
5. **Update Records** on the Yes path: update Cohort where `Id = {!$Record.Cohort__c}`, set `Status__c = Full`.
6. **Save** as "Enrolment Confirmed Actions", then **Activate**.

> **Test it:** change an Enrolment's Status to Confirmed → a Task appears on the record and filling CO-0001's last seat flips the Cohort Status to Full (UAT-06, UAT-07).

6.2 Screen flow — Quick_Enrol_Student (US-08)

Setup → Flows → New Flow → **Screen Flow**:

Element	Content
Screen 1 “Which course?”	Picklist using a **record choice set** on Course where `Active = true` (label: Name, value: Id)
Screen 2 “Which cohort?”	Radio buttons using a record choice set on Cohort where `Course = {!Course_Picklist}` AND `Status = Open`, label **Cohort Label**, sorted by Start Date
Screen 3 “Which student?”	**Lookup** screen component: Object API Name `Enrolment__c`, Field API Name `Student__c`
Get Records	Get the selected Course record (to pass its Name to the Apex action)
Action	Apex Action → **Find Available Cohorts** (CohortAvailabilityService) with Course Name — re-checks seats before writing
Decision	`{!Check_Seats.seatsAvailable} = true`?
Create Records (Yes)	Enrolment: Cohort = chosen cohort, Student = `{!Student_Lookup.recordId}`, Status = Confirmed
Screen (Yes)	Success confirmation showing the new record
Screen (No)	“Sorry — that cohort just filled up”, with Back
Fault paths	Every Get/Create/Action element gets a Fault connector to an error screen showing `{!$Flow.FaultMessage}`

Save as “Quick Enrol Student”, Activate. Surface it via App Launcher or a Quick Enrol button on the Cohort page.

6.3 Scheduled flow — Cohort_Start_Reminder (US-09)

1. New Flow → **Schedule-Triggered Flow** · Frequency **Daily** at 07:00.
2. **Get Records**: all Enrolments where `Status__c = Confirmed`.
3. **Loop** over them → **Decision**: `{!Loop.Cohort__r.Start_Date__c} = {!fReminderDate}`, where `fReminderDate` is a Date formula `{!$Flow.CurrentDate} + 3`.
4. On Yes → **Assignment**: build a Task (Subject “Reminder: your cohort starts in 3 days”, WhatId = enrolment Id, ActivityDate today, Priority High) and add it to a Task collection.
5. After the loop → Decision “any tasks?” → **Create Records** from the collection (one DML, outside the loop — the bulkification pattern in Flow).
6. Save as “Cohort Start Reminder”, Activate.

7 · Apex (Lab 4)

Create each class in VS Code (`force-app/main/default/classes`) or Developer Console (gear icon → Developer Console → File → New → Apex Class). Full source is in the GitHub repo — the design is summarised here.

7.1 EnrolmentTrigger — routing only

```
trigger EnrolmentTrigger on Enrolment__c (before insert, before update, after insert, after update) {
    if (Trigger.isBefore) {
        if (Trigger.isInsert)      { EnrolmentTriggerHandler.onBeforeInsert(Trigger.new); }
        else if (Trigger.isUpdate) { EnrolmentTriggerHandler.onBeforeUpdate(Trigger.new, Trigger.oldMap); }
    }
    if (Trigger.isAfter && (Trigger.isInsert || Trigger.isUpdate)) {
        EnrolmentTriggerHandler.onAfterChange(Trigger.new);
    }
}
```

7.2 EnrolmentTriggerHandler — all the logic

Method	Story	What it does
`setDefaults`	US-02	Defaults Enrolment Date to today, Status to Applied, Amount Paid to 0.
`buildUniqueKey`	US-05	Writes `Enrolment_Key__c = CohortId-StudentId` ; the unique external ID makes the database itself reject duplicates.
`preventOverbooking`	US-04	Bulk-safe capacity rule: collect cohort IDs → **one** SOQL query → allocate seats in memory → `addError()` on records that don't fit.
`enqueueStudentSummary`	US-13	Hands off to Queueable Apex after commit.

7.3 StudentSummaryQueueable

Aggregates each student's non-cancelled enrolments with one `GROUP BY` query and writes `Contact.Total_Enrolments__c` . Asynchronous = fresh governor limits, runs after the transaction commits.

7.4 CohortAvailabilityService & EnrolmentStatusService — the AI bridge

Both are `@InvocableMethod` classes: one method, three consumers (screen flow, Agentforce agent, REST). `findAvailableCohorts` returns open future cohorts with seats for a course-name search; `checkStatus` returns a student's enrolments by email. Every input/output has a clear `@InvocableVariable` description — the agent reads these to decide what to pass.

7.5 CohortController & EnrolmentController — the LWC layer

`@AuraEnabled(cacheable=true)` for reads (enables `@wire`), plain `@AuraEnabled` for the write. The write wraps DML in try/catch and rethrows `AuraHandledException` with a friendly sentence (NFR-04) — including translating the duplicate-key error into "That student is already enrolled on this cohort."

> **Checkpoint (US-04, US-05):** try to enrol a 4th student onto CO-0001 (capacity 3) — the save must fail with the "cohort is full" message. Enrol the same student twice — that must fail too.

8 · Lightning Web Components + Enrolment Console (Lab 5)

An LWC is three files: HTML template, JavaScript class, and a meta XML that says where it may be used. Create them in VS Code under `force-app/main/default/lwc` (right-click → SFDX: Create Lightning Web Component).

Component	Role	Key techniques
`cohortExplorer`	Course combobox + datatable of open cohorts	`@wire` to both controller reads; row selection dispatches a `cohortselected` CustomEvent; public `@api refresh()` calls `refreshApex`
`enrolmentPanel`	Contact picker + Enrol button	Receives cohort via `@api`; imperative Apex call; `ShowToastEvent`; `NavigationMixin` to the new record; fires `enrolled` event
`enrolmentConsole`	Wrapper wiring the two together	Data down (`cohort={selectedCohort}`), events up (`oncohortselected` , `onenrolled` → `explorer.refresh()`)

8.1 Create and activate the app page (manual step)

1. Setup → **Lightning App Builder** → the deployed page **Enrolment Console** already exists (or New → App Page → one region → drop **Enrolment Console** component on it).
2. Open it → **Activation…** → Lightning Experience tab → add to an app (e.g. Sales) → Save.
3. App Launcher → "Enrolment Console" → filter by course, select a cohort, pick a student, click **Enrol student**.

> **Checkpoint (US-10):** a toast appears, the seat count drops after refresh, and an overbooking attempt is still blocked by the Week-2 trigger — the console is just another channel over the same protected service layer.

9 · Agentforce agent (Lab 6 — manual, in the org)

> Agent metadata is not part of the CLI deployment — build it in Setup. If Agentforce is unavailable in your Developer org, the two invocable Apex actions are still deployed and the learning objective (the `@InvocableMethod` bridge) survives.

1. **Enable:** Setup → **Einstein Setup** → turn on Einstein → then Setup → **Agentforce Agents** (assign yourself the Agentforce permission set if prompted).
2. **Create the agent:** New Agent → type Employee/internal assistant · Name **Oxfordable Student Assistant** · Role: "Answers questions about our courses, cohort availability and a student's own enrolment status." · Company: "Oxfordable Careers, a technology training provider." · Tone: warm, concise, professional — never invent dates, prices or seat numbers.
3. **Create two actions** (Agent Actions → New → Apex):

Action	Apex	Input	Output
Find Available Cohorts	CohortAvailabilityService.findAvailableCohorts	Course Name (text)	Answer, Available Cohorts, Seats Available
Check Enrolment Status	EnrolmentStatusService.checkStatus	Student Email (text)	Answer, Found

   Write every description **for a colleague who has never seen your org** — thin descriptions are the number-one reason agents misbehave.
4. **Create the topic** "Course Enrolment Enquiries": classification = "Use this topic when someone asks about our courses, when a cohort starts, whether seats are available, how to join, or the status of an enrolment they already hold." Scope: answer only from the assigned actions. Instructions: confirm the course before searching · always state start date and seats remaining · offer the waiting list when full · ask for the booking email for status questions · never share one student's details with another · pass anything else to the team. Assign both actions.
5. **Test in the conversation preview:** "Do you have any Advanced cohorts with space?" → live list · "What is the status of my enrolment?" → asks for email · an off-topic question → graceful decline. After each test open the **reasoning trace** to see which topic and action fired.
6. **Activate** the agent.

10 · Testing (Lab 7)

Three test classes are deployed (all green in the target org):

Class	Covers
`TestDataFactory`	Reusable course/cohort/student/enrolment creation. Contacts are inserted with `DuplicateRuleHeader.allowSave = true` so org duplicate rules never break the tests. No hard-coded IDs, no org data.
`EnrolmentTriggerTest`	Defaults (US-02) · overbooking blocked (US-04, negative test with `Database.insert(rec, false)`) · **200-record bulk test** (NFR-01) asserting the capacity ceiling is never exceeded · duplicate blocked (US-05) · Queueable updates Contact (US-13, forced by `Test.stopTest()`)
`CohortServicesTest`	Both controllers, both invocable services, happy and error paths

```
sf apex run test --target-org <alias> --code-coverage --result-format human --wait 10
```

> **Two real-world lessons baked into the tests:** (1) `WITH SECURITY_ENFORCED` means the running user needs FLS — assign the permission set before running tests as yourself. (2) A 200-record partial-save with mixed successes triggers Salesforce retry behaviour; the bulk test asserts the true business invariant — the cohort can *never* be oversold.

11 · Security & deployment (Lab 8)

11.1 Permission set (NFR-05 — never edit profiles)

The deployed **Enrolment Console User** permission set grants: Read/Create/Edit on Course, Cohort, Enrolment · Read on Contact · FLS on all custom fields · access to the four Apex classes and the Quick Enrol flow · the three tabs visible. Assign it:

```
sf org assign permset --name Enrolment_Console_User --target-org <alias>
```

11.2 Version control

```
git clone https://github.com/dangutdavid/oxfodabledeveloper1_training.git
cd oxfodabledeveloper1_training
git checkout -b feature/my-change   # branch, change, commit, push, PR
```

11.3 Deploy to any org

```
# authorise
sf org login web --alias targetorg
# validate first - runs every check, saves nothing
sf project deploy start --source-dir force-app --target-org targetorg --test-level RunLocalTests --dry-run
# then deploy for real
sf project deploy start --source-dir force-app --target-org targetorg --test-level RunLocalTests
```

11.4 Post-deployment checklist

1. Confirm all three Flows are **Active** (Setup → Flows).
2. Assign the **Enrolment Console User** permission set to users.
3. Activate the **Enrolment Console** page and add tabs to app navigation (Section 8.1).
4. Build/activate the Agentforce agent and re-test one question (Section 9).
5. Deliverability = **All Email**.
6. Load reference data — **records are not deployed**: `sf apex run --file scripts/apex/seed.apex --target-org targetorg`.
7. Run smoke tests UAT-03, UAT-04, UAT-10 below.
8. Tag the release: `git tag v1.0 && git push --tags`.

12 · UAT script — prove it works

ID	Story	Test step	Expected result
UAT-01	US-01	Create a Course and a Cohort linked to it	Both save; the Cohort shows the Course
UAT-02	US-01	Set Cohort End Date before Start Date	Validation error shown
UAT-03	US-03	Create one enrolment on a capacity-3 cohort	Enrolled Count = 1, Seats Remaining = 2
UAT-04	US-04	Enrol a 4th student on a capacity-3 cohort	Save blocked with the custom capacity message
UAT-05	US-05	Enrol the same student twice on one cohort	Save blocked (duplicate key)
UAT-06	US-06	Change an enrolment Status to Confirmed	Email sent to the student; follow-up Task created
UAT-07	US-07	Fill the final seat on a cohort	Cohort Status becomes Full automatically
UAT-08	US-08	Run the Quick Enrol screen flow end to end	Enrolment created; confirmation screen shown
UAT-09	US-09	Cohort start date = today+3; run the scheduled flow	Reminder Task for confirmed enrolments only
UAT-10	US-10	Enrolment Console: filter, select, enrol	Toast shown; seat count refreshes; record created
UAT-11	US-11	Ask the agent “which Advanced cohorts have seats?”	Live cohorts with dates and seats remaining
UAT-12	US-12	Ask the agent for status with a known email	Cohort, start date and status returned
UAT-13	US-13	Check the student’s Contact record	Total Enrolments reflects active enrolments
UAT-14	US-14	Run all Apex tests	All pass; coverage ≥ 75%
UAT-15	US-14	Deploy to a second org; repeat UAT-03 & UAT-10	Both behave identically

Appendix · What is deployed where

Type	Components
Custom Objects	Course__c, Cohort__c, Enrolment__c (+ Contact.Total_Enrolments__c)
Validation Rules	End_Date_After_Start_Date, Payment_Not_Above_Fee
Flows (Active)	Enrolment_Confirmed_Actions, Quick_Enrol_Student, Cohort_Start_Reminder
Apex	EnrolmentTrigger, EnrolmentTriggerHandler, StudentSummaryQueueable, CohortAvailabilityService, EnrolmentStatusService, CohortController, EnrolmentController, TestDataFactory, EnrolmentTriggerTest, CohortServicesTest
LWC	cohortExplorer, enrolmentPanel, enrolmentConsole
Pages / Tabs	Enrolment_Console app page · Course, Cohort, Enrolment tabs
Security	Enrolment_Console_User permission set (assigned to maren.dangut@gmail.com)
Scripts	scripts/apex/seed.apex · manifest/package.xml

Appendix B · Reproduce this project yourself — with your own repo, org and AI assistant

This appendix is the **exact, copy-and-paste runbook** used to produce this project. You will **not** clone this repository — you will create your **own** repository and your **own** org, use Claude (or another AI coding assistant) to **generate all the code**, and then add, commit, push, deploy and test it yourself. Wherever you see a placeholder in `<angle-brackets>`, replace it with your own value before running the command.

Placeholder	Replace with	Example used in this build
`<your-github-username>`	Your GitHub username	dangutdavid
`<your-repo-name>`	Your new repository name	oxfodabledeveloper1_training
`<your-org-username>`	Your Developer Edition org username	maren.dangut@gmail.com
`<your-org-alias>`	A short nickname for the org	devorg

B.1 One-time setup — install and sign in (copy & paste)

```
# check the tools are installed
sf --version
git --version
gh --version          # GitHub CLI - install from cli.github.com if missing

# sign in to GitHub (one time)
gh auth login

# authorise your Salesforce org (opens a browser - log in as your org user)
sf org login web --alias <your-org-alias>

# confirm the org is connected
sf org list
```

B.2 Create YOUR OWN empty GitHub repository (copy & paste)

```
gh repo create <your-github-username>/<your-repo-name> --public
# verify it exists (it should be empty)
gh repo view <your-github-username>/<your-repo-name>
```

B.3 The prompt — have Claude generate ALL the code

Open Claude Code (or Claude with file attachments) in your working folder. **Attach these four course documents** to the conversation first:

1. Oxfordable_Salesforce_Participant_Workbook_Week1.pdf
2. Oxfordable_Salesforce_Participant_Workbook_Week2.pdf
3. Oxfordable_Salesforce_Participant_Workbook_Week3.pdf
4. Oxfordable_Salesforce_3Week_Programme_Handbook.pdf

Then paste this prompt **exactly**, with your own placeholders filled in:

```
i want you to read this attached materials – create salesforce project and write all
the code – creating the objects and fields needed, validation rules, pages, flows,
apex, agents, etc
commit and push to
https://github.com/<your-github-username>/<your-repo-name>

deploy to salesforce org  username –    <your-org-username>

after that create a pdf document with step by step for user to flow to create each.
```

That is the verbatim prompt that produced this project (typos and all — AI assistants cope fine). A cleaned-up version you can use instead:

```
Read the four attached Oxfordable workbooks/handbook. Build the complete CareerLaunch
Salesforce project as an SFDX source project: the Course__c, Cohort__c and Enrolment__c
objects with every field from the solution design (including the roll-up summary,
formula fields and unique external ID), both validation rules, the three flows
(Enrolment_Confirmed_Actions record-triggered, Quick_Enrol_Student screen flow,
Cohort_Start_Reminder scheduled), all Apex (EnrolmentTrigger + handler,
StudentSummaryQueueable, CohortAvailabilityService, EnrolmentStatusService,
CohortController, EnrolmentController, TestDataFactory + the two test classes),
the three LWCs and the Enrolment Console app page, the Enrolment_Console_User
permission set, a package.xml manifest and a seed.apex sample-data script.

Then: commit and push everything to https://github.com/<your-github-username>/<your-repo-name>,
deploy to my org <your-org-username>, assign the permission set to me, run all the
Apex tests and fix anything that fails, run the seed script, and finish with a
step-by-step PDF guide covering how to build each component by hand.
```

> **What to expect:** Claude will scaffold the project with `sf project generate --name careerlaunch`, write every file under `force-app/main/default`, deploy, run the tests, fix failures (see B.9), push to your repo and produce the PDF. Stay in the loop: read what it deploys, and compare the generated files against Sections 2–11 of this guide — reviewing generated code is the skill being trained.

B.4 What must exist afterwards — review checklist

Folder (under force-app/main/default)	Expected files
objects/	Course__c, Cohort__c, Enrolment__c (object + fields + validationRules), Contact/fields/Total_Enrolments__c
classes/	9 classes: trigger handler, queueable, 2 invocable services, 2 controllers, TestDataFactory, 2 test classes (+ .cls-meta.xml each)
triggers/	EnrolmentTrigger.trigger + meta
flows/	Enrolment_Confirmed_Actions, Quick_Enrol_Student, Cohort_Start_Reminder
lwc/	cohortExplorer, enrolmentPanel, enrolmentConsole
flexipages/ · permissionsets/ · tabs/	Enrolment_Console · Enrolment_Console_User · 3 custom tabs
manifest/ · scripts/apex/	package.xml · seed.apex

B.5 Deploy to your org (copy & paste)

```
cd careerlaunch

# deploy all components
sf project deploy start --source-dir force-app --target-org <your-org-alias> --wait 15

# assign the permission set to yourself BEFORE running tests
# (WITH SECURITY_ENFORCED needs field-level security on the new fields)
sf org assign permset --name Enrolment_Console_User --target-org <your-org-alias>
```

B.6 Run the tests (copy & paste)

```
sf apex run test --target-org <your-org-alias> \
  --tests EnrolmentTriggerTest --tests CohortServicesTest \
  --code-coverage --result-format human --wait 10
# expected: 10/10 Pass, class coverage 79-100%
```

B.7 Seed the sample data (copy & paste)

```
sf apex run --file scripts/apex/seed.apex --target-org <your-org-alias>
# creates 2 courses, 2 open cohorts (capacity 3 and 10), 3 student contacts
```

B.8 Add, commit and push to YOUR repository (copy & paste)

```
git init
git add .
git commit -m "CareerLaunch v1: data model, automation, Apex, LWC, flows, permission set"
git branch -M main
git remote add origin https://github.com/<your-github-username>/<your-repo-name>.git
git push -u origin main

# later changes follow the same rhythm:
git add .
git commit -m "describe your change"
git push
```

B.9 Fixes discovered during the real deployment (expect these, learn from these)

Symptom	Root cause	Fix
Contact inserts fail in tests with DUPLICATES_DETECTED	The org has an active Contact duplicate rule	TestDataFactory inserts contacts with `Database.DMLOptions.DuplicateRuleHeader.allowSave = true`
QueryException “secure query included inaccessible field”	`WITH SECURITY_ENFORCED` enforces the running user’s FLS; API-created fields grant FLS to nobody	Assign the Enrolment_Console_User permission set before running tests (step B.5)
CANNOT_EXECUTE_FLOW_TRIGGER on confirmed enrolments in tests	The flow’s Send Email action failed (deliverability/limits) and killed the save	Fault path on the email action so a failed email never blocks the enrolment
Bulk test expected exactly 150 successes, got fewer	Partial-save retries + the unique key make “exactly capacity” non-deterministic in one 200-record batch	Assert the true invariant: the cohort can never exceed capacity, and the overflow records are always blocked

B.10 Everyday commands you will keep using (copy & paste)

```
sf org open --target-org <your-org-alias>                              # open the org in a browser
sf project retrieve start --manifest manifest/package.xml \
  --target-org <your-org-alias>                                        # pull clicks-built changes into source
sf project deploy start --source-dir force-app/main/default/classes \
  --target-org <your-org-alias>                                        # redeploy just the Apex
sf apex tail log --target-org <your-org-alias>                         # stream debug logs
sf project deploy start --source-dir force-app --target-org <second-org> \
  --test-level RunLocalTests --dry-run                                 # validate-only against a 2nd org
git status ; git diff                                                  # see what changed
git tag v1.0 && git push --tags                                        # tag a release
```